# OpportunityKit — SETU 2026 Prototype Submission

SETU 2026 prototype submission · generated 17 October 2026

| | |
|---|---|
| **Project name** | OpportunityKit |
| **Problem statement** | Opportunity requirements are scattered across different sources and formats; students struggle to determin |
| **Team name** | [TO BE FILLED IN — see SUBMISSION-STATUS.md] |
| **Team member(s)** | [TO BE FILLED IN — see SUBMISSION-STATUS.md] |
| **Prototype status** | Working MVP demo. No paid API, external account or configuration required. |

## 1. Problem and proposed solution

Opportunity requirements are scattered across posters, PDFs, emails and chat groups, each in a different format. Students struggle to determine whether they meet the criteria, and missing skills, documents or unconfirmed information create uncertainty — which feels like rejection.
OpportunityKit structures requirements into a consistent format, compares them with the student profile, explains every result as MATCH, MISMATCH or MISSING INFORMATION, and turns gaps and unknowns into an actionable preparation checklist. The distinguishing principle — **"missing information is not the same as ineligibility"** — is demonstrated through a live interaction in the app, not just a tagline.

## 2. Demonstrated features

- One-click interactive demo with no registration and a clearly labeled sample opportunity
- Requirement extraction from opportunity descriptions (validated parsing; manual add supported)
- Three-state eligibility analysis: MATCH, MISMATCH, MISSING INFORMATION — each with an explanation
- Per-requirement "Why this status?" detail view (rule applied, profile value considered, source line)
- Preparation checklist generated from gaps and unknowns, with saved progress
- Profile editing with automatic re-analysis and change highlighting
- Demo reset, empty/loading/error states, responsive layout
- Automated tests covering extraction, the rules engine, the checklist and the full demo sequence (npm test)

## 3. Technology stack and technical approach

- Frontend: Next.js (App Router), React, TypeScript, Tailwind CSS — implemented.
- Requirement extraction: validated rule-based parsing with zod schemas; optional server-side LLM integration is PLANNED, not configured (/api/extract reports llmConfigured: false).
- Eligibility engine: explicit, documented, unit-tested rules (RULE-DEGREE-01 … RULE-OTHER-01).
- Persistence: localStorage for the demo; Supabase PostgreSQL is an optional integration, not configured.
- Validation: typed zod schemas, form validation, automated vitest tests.
- Deployment: see the Live Prototype link status below.

Pipeline: Student Profile + Opportunity Description → Requirement Extraction → Validated Structured Requirements → Eligibility Rules Engine → Explained Results → Preparation Checklist.

## 4. Demonstration overview

Landing page → Start Interactive Demo → fictional profile loads → open the labeled sample internship → extract requirements → run the analysis → see MATCH / MISMATCH / MISSING INFORMATION with explanations → open a requirement to see why → generate the checklist → mark an item complete (progress saved) → edit the profile (e.g. CGPA 7.2 → 8.0) and re-run to watch results change. Full timed script: docs/DEMO_VIDEO_SCRIPT.md.

## 5. Submission links

PENDING

### Live Prototype

(not available yet)

**PENDING.** Deploy the prototype (README.md → "Deploying"), then re-run this script with --live-url. Deploy target: Vercel or Netlify (free tier). Deploy the prototype (README.md → "Deploying"), then re-run this script with --live-url.

PENDING

### GitHub Source Code

(not available yet)

**PENDING.** Push this repository to GitHub (README.md → "Publishing the source code"), then re-run with --github-url. The complete source is in this repository (README.md has push instructions). Push this repository to GitHub (README.md → "Publishing the source code"), then re-run with --github-url.

PENDING

### Demonstration Video

(not available yet)

**PENDING.** Record and publish the demo video per docs/DEMO_VIDEO_SCRIPT.md, then re-run with --video-url. Script and recording checklist: docs/DEMO_VIDEO_SCRIPT.md. Record and publish the demo video per docs/DEMO_VIDEO_SCRIPT.md, then re-run with --video-url.

QR code for the live prototype will be included once the permanent URL exists.

## 6. Prototype limitations and demo-mode functionality

- Demo mode: all student, company and opportunity data is fictional. Persistence is localStorage only (demo storage); Supabase PostgreSQL is an optional integration that is not configured.
- The eligibility engine covers the requirement types in lib/types.ts; anything else is surfaced as MISSING INFORMATION for manual review — never silently passed or failed.
- No LLM, database or paid API is configured or operational. Everything advertised as "optional" is labeled as such in the app.
- No survey results, user statistics, performance metrics or validation evidence are claimed.

Regenerate this PDF after any URL changes: python3 scripts/make_pdf.py --team "..." --member "..." --live-url ... --github-url ... --video-url ... --out submission/OpportunityKit-SETU2026-Submission.pdf